LEAF-IIoT: Cryptanalysis, Flaws and Vulnerabilities

1 Cryptanalysis, Flaws and Vulnerabilities of LEAF-IIoT

1.1 1: Lack of Perfect Forward Secrecy (PFS)

1.1.1 Claim

The protocol does not provide Perfect Forward Secrecy (PFS); compromise of the gateway master secret allows reconstruction of past session keys.

1.1.2 Protocol-Based Verification

In LEAF-IIoT, the session key is defined as:

$$SK = H\big(ID_u \parallel SID_i \parallel N_u \parallel N_g \parallel N_d\big). \tag{1}$$

Observe the following facts:

1. N_u is transmitted in clear in **Msg1**.
2. N_g is generated by the gateway and transmitted encrypted under B_{kd}.
3. N_d is generated by the device and transmitted encrypted under B_{kd}.
4. B_{kd} is stored at the gateway encrypted under K_{GW}:

$$(C_a, Tag_a) = \mathcal{E}_{K_{GW}}(B_{kd} \parallel SID_i, RA_{ID}). \tag{2}$$

If an adversary compromises K_{GW} at time t, then:

$$B_{kd} = \mathcal{D}_{K_{GW}}(C_a, Tag_a), \tag{3}$$
$$(N_d, H(SK)) = \mathcal{D}_{B_{kd}}(C_{rep}, T_{rep}), \tag{4}$$
$$(ID_u, N_u, N_g) = \mathcal{D}_{B_{kd}}(C_{req}, T_{req}). \tag{5}$$

Hence, all components of SK for *past sessions* become computable:

$$SK^{(old)} = H(ID_u \parallel SID_i \parallel N_u \parallel N_g \parallel N_d). \tag{6}$$

1.1.3 Conclusion

The vulnerability is real and reproducible. LEAF-IIoT does *not* achieve Perfect Forward Secrecy.

Root Cause: The session key depends exclusively on nonces protected by long-term symmetric secrets. No ephemeral Diffie–Hellman or PUF-derived ephemeral secret contributes to SK.

Severity: High (standard violation in modern authentication and key agreement protocols).

1.2 2: Replay Attacks Within Timestamp Window

1.2.1 Claim

Timestamp-based freshness checks permit replay attacks within the accepted delay window.

1.2.2 Protocol-Based Verification

The gateway verifies freshness via:

$$|T_1^* - T_1| \leq \Delta T, \tag{7}$$

but **does not store any session state**, such as:

$$(ID_u, N_u, T_1). \tag{8}$$

Let an adversary record:

$$\text{Msg1} = \{DID_u, M_1, N_u, T_1\}.$$

Within ΔT, the adversary can replay Msg1 verbatim. The gateway will:

1. Recompute MPW^*,
2. Verify M_1 successfully,
3. Initiate a fresh session with the device.

1.2.3 Conclusion

The vulnerability is real under a stateless gateway model.

Severity: Medium.

Remark: This is a well-known weakness of timestamp-only freshness without nonce caching. It does not break secrecy but enables resource-exhaustion and session duplication attacks.

1.3 3: Gateway as a Single Point of Cryptographic Failure

1.3.1 Observation

Although the paper claims resistance to privileged insider attacks, the gateway is trusted with:

- Reconstruction of K_{GW},
- Decryption of all B_{kd} values,
- Access to all encrypted device credentials.

Compromise of the gateway implies:

$$\forall i: \; B_{kd}^{(i)} \text{ is recoverable.} \tag{9}$$

1.4 4: Lack of Explicit Key Confirmation

Theorem 1 (Incomplete Mutual Authentication)**.** *LEAF-IIoT does not provide explicit key confirmation from user to device.*

Proof. The protocol ends with user verifying $Ad_6 = H(SK_{uk})$, but no confirmation message is sent to ISD_i. Consider two scenarios:

1. **Case 1**: M_3 is modified but MAC passes (hash collision). ISD_i believes session is established, but user aborts.
2. **Case 2**: Network partition after M_3. ISD_i believes session is active, but user cannot send data.

Formally, let Accept_U and Accept_D be acceptance events. The protocol satisfies:

$$\Pr[\text{Accept}_U \neq \text{Accept}_D] > \mathsf{negl}(\lambda)$$

violating mutual authentication completeness. □

1.5 5: Biometric Information Leakage

Theorem 2 (Fuzzy Extractor Information Leakage)**.** *The helper data hdk leaks information about biometric template Bio_{uk}.*

Proof. From Dodis et al. (2008), for a (m, ℓ, t, ϵ)-fuzzy extractor:

$$\widetilde{H}_\infty(Bio_{uk}|hdk) \geq m - \ell - 2\log(1/\epsilon)$$

where $\widetilde{H}_\infty$ is average min-entropy.

For typical parameters $m = 200$ bits (biometric entropy), $\ell = 128$ bits (key length), $\epsilon = 2^{-80}$:

$$\widetilde{H}_\infty(Bio_{uk}|hdk) \geq 200 - 128 - 160 = -88 \text{ bits}$$

This negative bound indicates significant leakage. The actual leakage is:

$$I(Bio_{uk}; hdk) = H(Bio_{uk}) - \widetilde{H}_\infty(Bio_{uk}|hdk) \geq 200 - (-88) = 288 \text{ bits}$$

Thus, *hdk* reveals substantial information about Bio_{uk}. □

1.5.1 6: ASCON Nonce Misuse

Theorem 3 (ASCON Nonce Requirement Violation)**.** *LEAF-IIoT does not guarantee unique nonces for ASCON encryption under the same key.*

Proof. ASCON requires for each key K: $\forall i \neq j, N_i \neq N_j$ for encryptions $E_K^{N_i}$ and $E_K^{N_j}$.

In LEAF-IIoT:

- The nonce is implicitly derived from $A_1^* = H(ID_{uk} \| PW_{uk} \| B_k^*)$
- For the same user, ID_{uk}, PW_{uk}, B_k^* are constant until password change
- Thus, A_1^* and hence the nonce remains constant across sessions

This leads to nonce reuse. For ASCON, nonce reuse under same key allows:

$$C_1 = E_K^N(P_1)$$
$$C_2 = E_K^N(P_2)$$
$$\text{Then: } P_1 \oplus P_2 \text{ can be recovered}$$

breaking confidentiality. □

1.6 7: PUF Modeling Attack Feasibility

Theorem 4 (PUF Modeling Complexity)**.** *An arbiter PUF can be modeled with $O(n \log n)$ CRPs using logistic regression.*

Proof. An n-stage arbiter PUF implements:

$$R = \text{sign}\left(\sum_{i=1}^{n} w_i \phi_i(C) + w_{n+1}\right)$$

where w_i are weight parameters, $\phi_i(C)$ are challenge transformation.

Rührmair et al. (2013) show that with $m = O(n \log n)$ CRPs, the weights can be learned via logistic regression with success probability:

$$\Pr[\text{correct prediction}] \geq 1 - e^{-O(m/n)}$$

For $n = 64$ stages, $m \approx 64 \times \log_2(64) = 384$ CRPs suffice for 95% accuracy. Once modeled, the adversary can compute any $Res = PUF(Ch)$, compromising K_{GW}. □

1.7 8: Password Change Attack

Theorem 5 (Offline Dictionary Attack Complexity)**.** *The password change phase allows offline dictionary attack with complexity $O(|\mathcal{D}| \cdot C_{hash})$.*

Proof. Given stolen device with $\{C_a, MAC_1, hdk, hdk1\}$, adversary tests password guesses $PW' \in \mathcal{D}$:

```
for each PW' ∈ D do
    B'_k ← Rep(Bio_uk, hdk)          ▷ Using captured biometric or guessing
    A'_1 ← H(ID_uk ‖ PW' ‖ B'_k)
    (P'_a, MAC'_1) ← D_{A'_1}{C_a}
    if MAC_1 = MAC'_1 then return PW'
    end if
end for
```

Success probability after q queries:

$$\Pr[\text{Success}] = \frac{q}{|\mathcal{D}|}$$

With $|\mathcal{D}| = 10^6$ common passwords, expected queries: 5×10^5. □

1.8 9: PUF Challenge Injection

Theorem 6. *Finding Ch' such that $HD(PUF(Ch'), PUF(Ch)) \leq \epsilon$ requires $O(2^\epsilon)$ attempts.*

Proof. For ϵ-error tolerance, search space size:

$$|\mathcal{S}_\epsilon| = \sum_{i=0}^{\epsilon} \binom{n}{i}$$

Using approximation $\binom{n}{k} \approx 2^{nH(k/n)}$ where $H(p) = -p\log_2 p - (1-p)\log_2(1-p)$:

$$|\mathcal{S}_\epsilon| \approx 2^{nH(\epsilon/n)}$$

For $n = 256$, $\epsilon = 10$: $H(10/256) \approx 0.24$, so $|\mathcal{S}_\epsilon| \approx 2^{61.4}$. Thus, attack requires $O(2^{61.4})$ operations, which is feasible with parallel PUF evaluation. □

1.9 10: Fixed-Point Attack

Theorem 7. *Finding fixed-point x where $H(x) = x$ requires $O(2^{n/2})$ hash computations.*

Proof. By birthday paradox, for $H : \{0,1\}^n \to \{0,1\}^n$, finding fixed point requires checking $\sqrt{\pi/2} \cdot 2^{n/2}$ inputs.

For SHA-256 ($n = 256$): 2^{128} operations, which is computationally infeasible. However, the nested construction:

$$SK = H(H(\cdots) \oplus \cdots)$$

creates more complex algebraic structure that could reduce security margin. □

1.10 11: Resource Exhaustion

Theorem 8. *An attacker can exhaust gateway resources with N requests where $N \geq C_{max}/C_{auth}$.*

Proof. Let C_{auth} be authentication cost and C_{max} gateway capacity. Attacker sends N spoofed M_1 messages. Gateway performs:

$$\begin{aligned}
\text{Operations} &= N \times (C_{PUF} + C_{FE} + 2C_{ASCON} + C_{Hash}) \\
C_{PUF} &\approx 10^6 \text{ cycles} \\
C_{ASCON} &\approx 2 \times 10^3 \text{ cycles} \\
C_{FE} &\approx 10^5 \text{ cycles} \\
C_{Hash} &\approx 10^3 \text{ cycles}
\end{aligned}$$

Total: $\approx 1.1 \times 10^6$ cycles per authentication.

For 1 GHz processor: $N_{max} \approx 909$ authentications/second to saturate CPU. This is easily achievable by botnet. □

1.11 12: Timing Attack

Theorem 9. *ASCON timing variations enable key recovery with $O(\sigma^2/\Delta_t^2)$ samples.*

Proof. Let timing difference Δ_t between key-dependent paths have variance σ^2. Using statistical detection theory, number of samples needed for reliable bit detection:

$$N \geq \frac{8\sigma^2}{\Delta_t^2} \ln\left(\frac{1}{P_{error}}\right)$$

For $\sigma = 10$ ns (CPU noise), $\Delta_t = 2$ ns (carry propagation), $P_{error} = 0.01$:

$$N \geq \frac{8 \times 100}{4} \ln(100) \approx 200 \times 4.6 = 920 \text{ samples}$$

Thus, timing attack is practical. □